

NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING

PLACEMENT EXAMINATION FOR CG1101/CS1010/CS1010E Semester 1 AY2012/2013

PAPER 1

17 July 2012

Time allowed: 45 minutes

INSTRUCTIONS TO CANDIDATES

1. This examination paper consists of **TWENTY (20)** multiple-choice questions (MCQs) and comprises **TWELVE (12)** printed pages.
2. This is a **closed book** examination.
3. Answer all questions on the given OCR form.
4. Use **2B pencils** only for the OCR form.
5. In Section A box of the OCR form,
 - fill in **CG1101**, **CS1010** or **CS1010E** for “Module Code”, your “Contact Number” and “Signature”;
 - instead of “Academic Year/Semester”, write your **Full Name** there;
 - you may ignore “Date”.
6. In Section B box of the OCR form,
 - **write** your **NUS Student Number**, and
 - **shade** the corresponding bubbles for your **Student Number**.

If you have not obtained your NUS Student Number, write your **NUS Application Number** in the space on the right of Section B box.
7. In Section C box of the OCR form, shade one bubble for each question.
 - Each question has only one correct answer and is worth one mark. There is no penalty for wrong answer.
 - Shading more than one bubble for a question will invalidate your answer.

B 1. What is the output of the following code fragment?

```
int a = 0, b = 1000, count = 0;
while (8*a < b) {
    a += 2;
    b /= 2;
    count++;
}
printf("%d\n", count);
```

Handwritten notes for the while loop:

Iteration	a	b	8*a < b
1	2	500	16 < 500 ✓
2	4	250	32 < 250 ✓
3	6	125	48 < 125 ✓
4	8	62	64 < 62 ✗

- (A) 3
- (B) 4
- (C) 6
- (D) 7
- (E) 8

C 2. What is the output of the following code fragment, given that the value of **sqrt(35)** is **5.916080**?

```
float x = (int) sqrt(35) * (int) sqrt(35);
float y = (int) sqrt(35) * sqrt(35);
float z = (int) (sqrt(35) * (int) sqrt(35));
printf("%.2f %.2f %.2f\n", x, y, z);
```

Handwritten note: $\sqrt{35}$

- (A) 36.00 35.50 35.00
- (B) 36.00 35.00 35.00
- (C) 25.00 29.58 29.00
- (D) 25.00 29.00 29.00
- (E) None of the above.

E 3. Given the following code fragment, which set of input and output is incorrect?

```
int a, b, c;
scanf("%d %d", &a, &b);
c = ((a < b) || (--a < ++b)) ? --a : b++;
printf("%d %d %d\n", a, b, c);
```

- (A) Input: 1 2 → Output: 0 2 0 $1 < 2$ ✓ $0 \ 2$
- (B) Input: 2 1 → Output: 0 2 0 $2 > 1$ ✗ $1 < 2$ ✓ $0 \ 2$
- (C) Input: 3 1 → Output: 2 3 2 $3 > 1$ ✗ $2 < 2$ ✗ $2 \ 3$
- (D) Input: 3 3 → Output: 1 4 1 $3 < 3$ ✗ $2 < 4$ ✓ $1 \ 4$
- (E) None of the above.

Notes:

--a, a--, ++a, a++ will have side effect **after** it is executed. Short circuit execution is adopted in the C language. For && of many conditions to be false, one false is already enough. For || of many conditions to be true, one true is already enough. But for && of many conditions to be true, all its conditions must all be true. Also, for || of many conditions to be false, all its conditions must all be false.

4. Given the following code fragment, what inputs generate no output?

```
int a, b, c;
printf("Enter a, b, c: ");
scanf("%d %d %d", &a, &b, &c);

if (a > b)
    if (b > c)
        printf("X");
    else
        printf("Y");
else
    if (b > c)
        printf("Z");
```

- (i) Enter a, b, c: 1 2 3 ✓
- (ii) Enter a, b, c: 3 2 1 ✗
- (iii) Enter a, b, c: 5 5 7 ✓
- (iv) Enter a, b, c: 6 2 4 ✗
- (v) Enter a, b, c: 9 9 9 ✓

- (A) Only (i), (iii) and (v)
- (B) Only (i), (iv) and (v)
- (C) Only (ii), (iii) and (v)
- (D) Only (i), (iii), (iv) and (v)
- (E) Only (ii), (iii), (iv) and (v)

5. What is the output of the following code fragment?

```
int a = 1, b = 9, sum = 0;

while (a = b || a <= 100) {
    sum++;
    a++;
}

printf("%d\n", sum);
```

- (A) 99
- (B) 100
- (C) 101
- (D) The program cannot be compiled.
- (E) No output as there is an infinite loop.

For questions 6 and 7, please refer to the following partial program.

```
typedef struct {
    int x, y;
    double length;
} Point;

// function prototype omitted

int main(void) {
    Point p;

    printf("Enter x- and y-coordinates: ");
    // scanf statement here

    compute_length(&p);
    printf("length = %.3f\n", p.length);

    return 0;
}

// function definition omitted
```

6. Which of the following **scanf()** statements work with the given program?

- (i) `scanf("%d %d", &p.x, &p.y);`
- (ii) `scanf("%d %d", &(p.x), &(p.y));`
- (iii) `scanf("%d %d", p.x, p.y);`
- (iv) `scanf("%d %d", p->x, p->y);`
- (v) `scanf("%d %d", &p->x, &p->y);`

- (A) Only (i) and (ii)
- (B) Only (i) and (iv)
- (C) Only (ii) and (iv)
- (D) Only (i), (iii) and (iv)
- (E) Only (ii), (iv) and (v)

7. If the correct **scanf()** statement has been added, which of the following **compute_length()** functions would work with the given partial program? You are not to modify any part of the given code.

(A)

```
void compute_length(Point *p) {
    int x = p.x;
    int y = p.y;
    p.length = sqrt(x*x + y*y);
}
```

(B)

```
void compute_length(Point p) {
    int x = p->x;
    int y = p->y;
    p->length = sqrt(x*x + y*y);
}
```

(C)

```
void compute_length(Point *p) {
    int x = p->x;
    int y = p->y;
    p->length = sqrt(x*x + y*y);
}
```

(D)

```
void compute_length(Point *p) {
    int x = (*p)->x;
    int y = (*p)->y;
    (*p)->length = sqrt(x*x + y*y);
}
```

(E) None of the above.

8. Given the following program, what is the output?

```
#include <stdio.h>
int f(int, int);

int main(void) {
    int a = 2, b = 3;
    a = f(f(a, b), f(b, a));
    printf("%d %d\n", a, b);

    return 0;
}

int f(int a, int b) {
    b *= 3;
    return a + b;
}
```

- (A) 44 3
- (B) 44 9
- (C) 42 3
- (D) 38 3
- (E) None of the above

9. Given the following program, what is the output?

```
#include <stdio.h>
int f(int, int *);

int main(void) {
    int a = 2, b = 3;
    a = f(f(a, &b), &b);
    printf("%d %d\n", a, b);

    return 0;
}

int f(int a, int *b) {
    *b *= 3;
    return a + *b;
}
```

- (A) 20 3
- (B) 20 9
- (C) 38 3
- (D) 38 9
- (E) 38 27

- C 10. Given the following function, what is **f(8)**?

```
int f(int n) {
    if (n < 3) return n;
    else      return f(n-1) + 2 * f(n-3);
}
```

- (A) 8
(B) 15
(C) 36
(D) 64
(E) 128

$$\begin{aligned}
 f(8) &= f(7) + 2f(5) \\
 &= f(6) + 2f(4) + 2(f(3) + 2f(1)) \\
 &= f(5) + 2f(3) + 4f(1) + 2f(2) \\
 &= f(4) + 2f(2) + 4f(1) + 2f(1) \\
 &= f(3) + 2f(1) + 4f(1) + 2f(1) \\
 &= f(2) + 2f(1) + 4f(1) + 2f(1) \\
 &= f(1) + 2f(1) + 4f(1) + 2f(1) \\
 &= 1 + 2 + 4 + 2 = 9
 \end{aligned}$$

- B 11. Given the following function, what is **g(13)**?

```
int g(int n) {
    if (n < 3) return n;
    else      return (n/2) + g(n-3);
}
```

- (A) 9
(B) 17
(C) 20
(D) 23
(E) 25

$$\begin{aligned}
 g(13) &= 6 + g(10) \\
 &= 6 + 5 + g(7) \\
 &= 11 + 3 + g(4) \\
 &= 14 + 2 + g(1) \\
 &= 17
 \end{aligned}$$

- C 12. What is the output of the following code fragment?

```
int i, x = 0;

for (i = 1; x < 30; i++) {
    x += i;
    i++;
}

printf("%d\n", i);
```

- (A) 9
(B) 10
(C) 13
(D) 30
(E) 36

13. What is the output of the following code fragment?

```
char str[10] = "ABCDEFGHI";
int len = strlen(str);
int i;

for (i = len/2; i <= len; i++)
    str[i-2] = str[i];

printf("%s\n", str);
```

- (A) ABCDEF
- (B) ABCDEFG
- (C) ABCFGHI
- (D) ABDEFHG
- (E) ABEFGHI

14. What is the output of the following code fragment?

```
char days[][10] = { "Monday", "Tuesday", "Wednesday",
                    "Thursday", "Friday", "Saturday",
                    "Sunday" };

int i;

for (i = 0; i < 6; i++)
    if (strcmp(days[i], days[i+1]) < 0)
        strcpy(days[i], days[i+1]);

for (i = 0; i < 7; i++)
    printf("%c", days[i][2]);
printf("\n");
```

- (A) neduitn
- (B) ueehauu
- (C) eddutttn
- (D) eddutttn
- (E) neuitttn

15. Study the following program.

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *inf;
    char ch;
    int sum = 0;

    inf = fopen("data.in", "r");

    while ((ch = → get a charactergetc(inf)) != '\n') {
        toupper(ch); ←
        if (isupper(ch))
            sum++;
    }

    printf("%d\n", sum);
    fclose(inf);
    return 0;
}
```

Notes: The output of toupper(ch) is not received by any identifier thus it does not change anything. The value of ch has no change.

The content of the text file "data.in" is given below:

```
On the 1st day of Christmas
my true love sent to me:
A Partridge in a Pear Tree
```

What is the output of the program?

- (A) 2
- (B) 6
- (C) 61
- (D) 62
- (E) 77

16. Given the following program, what sorting technique does the **sortArray()** function implement, and what is the output?

```
#include <stdio.h>
void sortArray(int[], int);
void printArray(int[], int);

int main(void) {
    int arr[9] = { 3, 2, 8, 9, 1, 7, 6, 5, 4 };
    sortArray(arr, 9);
    return 0;
}

void sortArray(int a[], int size) {
    int i, j, min, temp;

    for (i = 0; i < size-1; i++) {
        min = i;
        for (j = i+1; j < size; j++) {
            if (a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
        if (i == 3)
            printArray(a, size);
    }
}

void printArray(int a[], int size) {
    int i;

    for (i = 0; i < size; i++)
        printf("%d ", a[i]);
    printf("\n");
}
```

- (A) Selection sort; Output: 1 2 3 9 8 7 6 5 4
- (B) Selection sort; Output: 1 2 3 4 8 7 6 5 9
- (C) Selection sort; Output: 1 2 3 4 5 7 6 8 9
- (D) Bubble sort; Output: 2 3 1 5 4 6 7 8 9
- (E) Insertion sort; Output: 1 2 3 4 8 9 7 6 5

17. What is the output of the following code fragment?

```
float arr[] = { 1.2, 2.3, 9.0, 8.8, 6.3, 2.1 };

printf("%d\n", sizeof(arr)/sizeof(arr[0]));
```

- (A) 3
- (B) 4
- (C) 6
- (D) 24
- (E) 32

18. Given the following program, what is the output?

```
#include <stdio.h>
#define LIMIT 10

int main(void) {
    int a[LIMIT] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    int i;

    for (i = 0; i < LIMIT; i++) {
        if (a[i] % 2)
            a[i] *= 2;
        else
            a[i] /= 3;
    }

    for (i = 0; i < LIMIT; i++)
        printf("%d ", a[i]);
    printf("\n");

    return 0;
}
```

- (A) 2 6 6 12 10 18 14 24 18 30
- (B) 1 4 1 8 2 12 3 16 3 20
- (C) 0 4 1 8 1 12 2 16 3 20
- (D) 2 1 6 2 10 2 14 3 18 4
- (E) 2 0 6 1 10 2 14 2 18 3

19. Given the following program, what is the output?

```
#include <stdio.h>
#define SIZE 5
int main(void) {
    int matrix[SIZE][SIZE];
    int r, c, i, sum;

    for (r = 0; r < SIZE; r++)
        for (c = 0; c < SIZE; c++)
            matrix[r][c] = r*r + c*c;

    sum = 0;
    for (i = 0; i < 3; i++)
        sum += matrix[2*i][i];
    printf("%d\n", sum);

    return 0;
}
```

- (A) 5
- (B) 14
- (C) 20
- (D) 25
- (E) 40

20. Study the program given below.

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    printf("%d; %s\n", argc, argv[3]);
    return 0;
}
```

The above program is compiled into the executable file **a.out**. What is the output if the program is run as follows?

a.out "The C programming language" was invented in 1972.

- (A) 5; invented
- (B) 6; invented
- (C) 6; in
- (D) 8; programming
- (E) 9; programming

END OF PAPER